

REMORA: Real-time Expressive Motion to Output Routing Audio

Timothée Dravet

Polytech Marseille, Aix-Marseille Université
France

timothee.dravet@etu.univ-amu.fr

Daniel Overholt

CREATE, Aalborg University
Denmark

dano@create.aau.dk

Abstract

REMORA turns a martial arts staff (bō) into a musical instrument: spins, tilts, and strikes become sound in real time. A wireless sensor unit on the staff tracks its motion and sends it to a companion audio plugin, which we built around a simple idea - instead of hard-wiring one fixed gesture-to-sound behaviour, the mapping itself is a small patch of connected building blocks that the performer can see and rewire live, on a visual canvas inside the plugin, much like a modular synthesiser. We show that this same small set of building blocks is enough to reproduce, as editable patches, everything from simple tilt-controlled pitch to compound movement-quality mappings inspired by Laban Movement Analysis. This paper describes the instrument’s hardware, its self-calibration method, the mathematics behind turning raw orientation into musical control, and the live patching interface itself, situates it against prior staff-shaped, baton, and visual-patching instruments, and - grounded in implementation-level evidence together with a protocol for the performer evaluation still to be run - reflects on what it means to make an instrument’s gestural vocabulary an explicit, editable part of the instrument.

CCS Concepts

- **Applied computing** → **Sound and music computing**;
- **Software and its engineering** → *Visual languages*.

Keywords

digital musical instrument, gestural control, inertial sensing, Laban Movement Analysis, OSC, staff instrument

1 Introduction

Motivation and Background

REMORA grew out of the first author’s parallel practice in martial arts and music, and a frustration shared by many hand-built digital musical instruments: once a gesture vocabulary is designed and coded, changing it means rebuilding the instrument rather than performing with it. That friction is the paper’s real starting point - not which single mapping best suits a staff, but whether a staff instrument’s gestural vocabulary could instead be something the performer authors and revises live, the way a modular-synthesis patch is built and rebuilt on a rack.

The bō staff itself sits in a gap the New Interfaces for Musical Expression (NIME) community has left largely unexplored: a full-body, swung, two-handed controller with its own rotational inertia and reach, distinct from the small handheld or body-mounted objects most inertial-sensor Digital Musical Instruments (DMIs) use.

Pairing that underused controller form with a mapping layer that can be reshaped at performance time, rather than fixed at design time, is the combination this paper investigates.

Objectives of the Study

This article supports four claims, each argued based on the system as it currently operates rather than being proposed as future work:

- (1) A small dataflow language - source, math, and sink node primitives - is embedded directly in the audio plugin and evaluated once per audio block as a directed acyclic graph (DAG); the performer rewires it live, without recompiling any code.
- (2) That same handful of primitives is expressive enough to reproduce eleven previously hand-coded mapping strategies as ordinary presets: the mapping design space explored by hand over the course of the project turns out to be spanned by a fairly compact algebra of building blocks.
- (3) A magnetometer-free, world-frame heading estimate (“facing”) is derived purely from the calibrated orientation stream, eluding the magnetic interference a stage rig would otherwise cause - a claim we bound to session lengths on the order of tens of minutes, since uncorrected gyroscopic yaw drift is expected to accumulate over longer, uninterrupted play (§7).
- (4) A mount-agnostic calibration transform lets the same instrument logic run correctly regardless of where the sensor is physically mounted on the staff.

Scope and Applications

REMORA targets solo live performance with a single staff and a single Inertial Measurement Unit (IMU) - not an installation piece, not a multi-performer controller (at least not yet, see Future Improvements). Worth being upfront about what’s out of scope for this paper too: no formal user study, no controlled comparison against other staff instruments, and no claim that patching a mapping is easier to learn than a fixed one - only that the capability exists and is expressive.

2 Related Work

Staff, Rod, and Baton DMIs

The T-Stick [4] is the closest prior instrument to REMORA’s physical form: a family of rod-shaped controllers sensing finger position and pressure along the shaft via capacitive touch, together with accelerometer-derived shake and tap gestures. Crucially, the T-Stick project pairs that sensing with an external, reconfigurable mapping layer (the digital-orchestra/libmapper tools built alongside it), so the gesture vocabulary is not permanently fixed to the hardware either - but that rewiring happens in supporting software outside the instrument, authored by a mapping designer between pieces,



This work is licensed under a Creative Commons Attribution 4.0 International License.

rather than as a performer-facing surface embedded in the instrument itself during play. REMORA differs from the T-Stick on two axes at once: its full-body two-handed swung form (closer to a bâ or fighting staff than a handheld rod), and a live-patchable mapping graph built directly into the audio plugin the performer plays through, rather than a separate mapping tool.

Conducting batons form a second, older lineage of rod-shaped expressive controllers. Marrin and Paradiso’s Digital Baton [5] combined accelerometers, infrared position tracking, and a pressure-sensitive tip to capture continuous conducting gesture, and the companion Conductor’s Jacket [6] extended the same goal - recording expressive, continuous musical gesture rather than discrete note events - to a wearable, full-body electromyography (EMG) and motion rig. Both systems used a fixed mapping, hand-engineered by the researchers for conducting gesture specifically; REMORA shares the goal of capturing continuous, qualitative gesture (via Laban Effort features, §4 and Table 1) but treats the mapping from that gesture to sound as something the performer edits rather than something fixed by the instrument builder in advance.

Swung or spun full-body objects are a comparatively thin category at NIME: the clearest match is Nam’s Musical Poi (mPoi) [7], a sensor-augmented poi that generates sound from swing motion. mPoi confirms that spun/swung objects are a recognised, if under-explored, DMI form, but it is a compact, single-mapping instrument without a reconfigurable mapping layer or a movement-quality vocabulary comparable to Laban Effort. REMORA occupies the same underexplored niche of full-body, swung, two-handed controllers but combines it with the T-Stick’s ambition of a reconfigurable gesture vocabulary and the conducting-baton lineage’s focus on continuous expressive quality rather than discrete triggers - a combination we are not aware of in prior staff-, rod-, or swung-object DMIs.

Inertial Sensing and Visual Patching in NIME

Visual dataflow patching is a much older paradigm than any of the staff instruments above. Max originated as Puckette’s Patcher [8] and has been maintained and extended commercially for over three decades, a history Puckette himself traces in “Max at Seventeen” [10]; Pure Data [9] is its open-source sibling, built around the same boxes-and-patch-cords model of unit generators wired together and rewired live. VCV Rack [1] is a more recent, community-maintained open-source descendant of the same idea applied to virtual modular synthesis. REMORA’s node-graph editor (§4) draws directly on this patching lineage - typed nodes, typed wires, live rewiring without stopping audio - but repurposes it specifically as a *gesture-to-sound mapping layer* rather than a general-purpose signal-processing environment: REMORA’s nodes represent motion features, shaping functions, and synthesis-parameter sinks, not arbitrary digital signal processing (DSP), and the graph lives inside a single instrument’s plugin rather than functioning as a standalone environment a performer would build an entire instrument in.

Using Laban Movement Analysis as a source of mapping features has real precedent in NIME and related venues. Camurri et al.’s EyesWeb [3] established a technical template for extracting expressive movement qualities from full-body motion in real time, explicitly grounded in Laban’s Effort theory, for interactive dance

and music systems. Bennett et al.’s DAMPER [2] used Laban’s four Effort factors directly as the design basis for a force-feedback interface platform, Ward et al. [12] used Laban Effort - Flow and the Recuperation phase of Effort phrasing in particular - as an analytic lens on theremin performers’ movement to inform instrument design, and Schacher [11] argues explicitly for Laban-derived motion/effort elements as a systematic basis for gesture-to-sound mapping in interactive dance. REMORA continues this line by computing the same four Effort qualities (Weight, Time, Space, Flow; Table 1) as real-time IMU-derived features, but where the cited work treats Effort primarily as an analysis or fixed-mapping-design tool, REMORA exposes those qualities as first-class typed nodes a performer can wire into an arbitrary sink at performance time - combining EyesWeb’s Effort-feature-extraction approach with the Max/Pd/VCV-style repatchable dataflow paradigm inside a single embedded, performer-facing instrument, which is the specific combination we have not found addressed together in prior work.

3 Design and System Architecture

Initial Design Requirements

Five requirements shaped the design from the outset, chosen to keep the instrument performable rather than merely functional on a workbench:

- **Full-body, swingable staff form factor.** Not a handheld controller: the instrument must support the reach, weight, and two-handed grip of an actual bâ, not a miniaturised stand-in for one.
- **Wireless sensing during performance.** A cable is acceptable only for charging between sets; nothing should tether the performer to a fixed point on stage.
- **Live-editable mapping.** Switching or rewiring the mapping graph must not interrupt audio output, so a change can be made mid-rehearsal without stopping the sound.
- **Mount-agnostic calibration.** Instrument behaviour must not depend on precisely where the sensor is glued to the staff, since exact placement varies from build to build and cannot be guaranteed reproducible by hand.
- **Low end-to-end latency.** The staff has to feel driven by the performer’s motion, not lagging behind it, which bounds how much time sensing, transport, and synthesis are together allowed to take.

Architecture Overview

REMORA’s architecture has two parts, matching the physical split between staff and host: firmware handles sensing and transport, and the plugin handles everything downstream of the incoming Open Sound Control (OSC) stream, from calibration through to the audio it produces. Figure 1 lays out that split directly, left to right: IMU sampling and OSC transmission on the staff, feeding the plugin’s own reception-to-synthesis pipeline on the host.

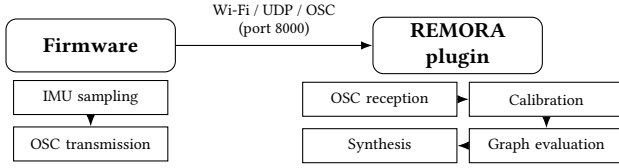


Figure 1: REMORA's signal flow, from IMU sampling on the staff to synthesis on the host.

4 Implementation

The Build

The sensor unit sits in a small OpenSCAD-parametrised enclosure, sized to the bô's diameter rather than assuming a fixed one. Because calibration is mount-agnostic, the enclosure can be placed anywhere along the staff; in practice it is usually mounted near one extremity, out of the way of the performer's grip. It is secured with three zip ties passing through channels in the casing - no adhesive or drilling - so it can be repositioned or removed freely. The same zip ties hold the two enclosure halves closed, together with screws that additionally clamp the internal electronics in place so nothing shifts under swing.

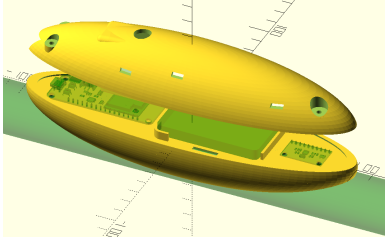


Figure 2: The parametric OpenSCAD enclosure, sized to the staff diameter and secured with zip ties.

Hardware

Inside the enclosure, a SparkFun ESP32-S2 Thing Plus reads a 9-degrees-of-freedom (9-DOF) MPU-9250 IMU over the Inter-Integrated Circuit (I2C) bus at 100 Hz and packs orientation, acceleration, and gyroscope readings into an OSC packet sent over the User Datagram Protocol (UDP) on port 8000. The firmware joins the host computer's own Wi-Fi hotspot directly rather than a dedicated access point, which keeps setup to a single pairing step at the cost of the range and interference sensitivity Discussion returns to (§7).

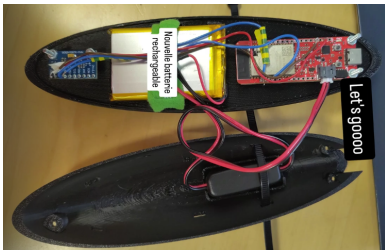


Figure 3: Component arrangement inside the enclosure - ESP32-S2, MPU-9250 IMU, and LiPo battery, wired as described in Hardware.

Software: Calibration and Motion Mathematics

Every equation below comes straight from the running code, each one is tagged with the function it lives in.

Orientation and Calibration. Rotation. Orientation is tracked as a unit quaternion (no gimbal lock); rotating v by q gives $v' = q v q^{-1}$, with q^{-1} the conjugate. Euler angles are derived only once, at the end, for the handful of mappings that still need them.

[MathHelpers::rotate] [conjugate]
[rotateVectorByQuaternion] [toEuler] [multiply]

Calibration, stage 1 - which axis is "forward". The sensor can be glued onto the staff at any orientation, so each candidate axis $L \in \{\pm X, \pm Y, \pm Z\}$ is scored against three reference poses A, B, C (quaternions q_A, q_B, q_C , chosen so forward varies substantially between them while up/right do not):

$$v_P = \widehat{\text{rotate}(L, q_P)}, \quad e(L) = |v_A \cdot v_B| + |v_B \cdot v_C| + |v_C \cdot v_A|$$

A low $e(L)$ (near-orthogonal v_A, v_B, v_C) marks the genuine forward axis; restricted to right-handed candidates,

$$L^* = \arg \min_{L: (v_A \times v_B) \cdot v_C < 0} e(L)$$

gives the alignment quaternion q_{align} .

[computeAlignQuat]

Stage 1 failure modes. computeAlignQuat reports failure when the three poses are too close together or nearly coplanar to fix a physical axis with confidence, and the User Interface (UI) surfaces this as a "Calibration failed, try again" prompt.

Calibration, stage 2 - fixing the twist. The sensor could still be rotated around L^* . The measured forward/up/right triad and the ideal instrument-frame triad are each orthonormalized via Gram-Schmidt into U and W ; the aligning rotation is the orthogonal-Procrustes closed form

$$R = WU^T,$$

converted to a quaternion q_{corr} via Shepperd's method.

[computeCorrection] [MathHelpers::fromMatrix]

Runtime. Every live reading becomes instrument space via $q' = q_{\text{corr}} (q_{\text{raw}} q_{\text{align}})$ - independent of where the sensor sits on the staff.

[REMORAProcessor::getCalibratedQuat]

Motion Feature Extraction. Everything below runs once per received packet, inside the shared analyzer that every mapping graph reads from - so no individual mapping repeats this work. The smoothing rates, thresholds, and envelope times named throughout this subsection were tuned by ear against the physical instrument during iterative testing, not derived analytically or fit to recorded data; see Discussion for the resulting lack of a perceptual evaluation of these choices.

[StaffMotionAnalyzer::computeFrame]

Table 1: Per-packet motion features.

Feature	Purpose
Timestep Δt	reject implausible packet gaps
Tip vector / rotation axis	derive tip velocity & spin axis
Gravity / dynamic accel.	separate gravity from motion
Laban Weight	movement forcefulness
Laban Time	suddenness of rotation-speed change
Laban Space	spin-axis consistency (focus)
Laban Flow	bound vs. free movement
Spin classification	vertical/horizontal spin & direction
Facing	east-west vs. north-south heading
Continuous spin count	count full turns per spin
Axial thrust	detect stab/strike along staff axis

Timestep. Δt is the time between two packets, clamped to a 0.1-200 ms valid range so a dropped or duplicated packet can't send the integrators below flying.

[calculateDeltaTime]

Tip vector and rotation axis. The tip direction $p_n = \text{rotate}((1, 0, 0), q_n)$ is buffered over the last three samples to derive tip velocity and, from that, the axis the staff is currently spinning around.

[updateTipPositionHistory] [calculateRotationAxisAtMidpoint]

Gravity and dynamic acceleration. The accelerometer always mixes gravity in with whatever motion the performer is making. To separate the two, REMORA keeps a running estimate of which way “down” currently is, by smoothing the inverse-rotated world +Z axis:

$$g_n = \text{onePole}(g_{n-1}, \text{rotate}(+Z, q_n^{-1}), \alpha=0.10)$$

Subtracting this gravity estimate from the raw reading leaves just the motion: $a_{\text{dyn}} = a_{\text{raw}} - g$.

[updateGravityVector] [calculateDynamicAcceleration]

Laban features. All four Laban Effort qualities reduce to the same shared one-pole smoother,

$$y_n = y_{n-1} + \alpha (x_n - y_{n-1}), \quad \alpha \in (0, 1],$$

applied to a derivative of either dynamic acceleration or gyroscope magnitude: Weight integrates dynamic acceleration into a leaky velocity estimate (its magnitude, fast-attack/slow-release enveloped) to capture forcefulness; Time is the clamped-positive rate of change of raw gyroscope magnitude, capturing suddenness; Space is the smoothed absolute dot product between consecutive rotation-axis directions, capturing how consistently the staff spins around the same axis versus wandering; Flow is the smoothed, clamped jerk of dynamic-acceleration magnitude, split into flow_bound (controlled) and $1 - \text{flow_bound}$ (free).

[MathHelpers::applyOnePoleFilter]

[integrateVelocityForLabanWeight] [updateLabanWeight]

[updateLabanTime] [updateLabanSpace] [updateLabanFlow]

Spin classification. Is the staff spinning like a baton (vertical) or more like a fan blade (horizontal)? Let $(\text{axis}_x, \text{axis}_y, \text{axis}_z)$ be the $\alpha=0.35$ one-pole-smoothed rotation axis from above. The spin is classified vertical when

$$\sqrt{\text{axis}_x^2 + \text{axis}_y^2} \geq |\text{axis}_z|,$$

i.e. when the horizontal-plane component outweighs the z component. Spin direction (clockwise/counter-clockwise, CW/CCW) then

comes either from the sign of the dominant in-plane axis component, or - in the Martial mappings - from the sign of that axis's dot product against a reference heading latched once, the first time a qualifying vertical spin happens after the mapping loads.

[updateSpinClassificationByAbsoluteComponent]

[updateSpinClassificationByReferenceAzimuth]

Facing. Rather than compute a compass heading - which would need a magnetometer, and stages are full of magnetic interference - REMORA asks a simpler question: is the staff pointing more east-west or more north-south? Let $(\text{dir}_x, \text{dir}_y)$ be the horizontal-plane components of whichever direction is currently relevant - the rotation axis during vertical spins, the tip direction during horizontal ones. Facing is then just a hysteresis comparison,

$$|\text{dir}_x| \geq 1.15 \cdot |\text{dir}_y|,$$

where the 1.15 factor gives asymmetric switch points ($\approx 48^\circ/42^\circ$) so the classification doesn't flicker right at the boundary.

[updateFacingClassification]

Continuous spin count. Full turns in the current spin, from $\|\omega\| \cdot \Delta t$ accumulated to 360° per lap; resets whenever the spin classification changes.

[accumulateContinuousSpins]

Axial thrust detection. Flags a stab/strike when the jerk of the tip-projected dynamic acceleration crosses zero as its magnitude passes 1.5g, gated against fast spins ($\|\omega\| < 90^\circ/\text{s}$) and cooled down between hits.

[detectAxialThrustPeaks]

Sensitivity of tuned constants. None of the constants above were fit to data; each biases the instrument in a specific direction. The gravity smoother's $\alpha=0.10$ trades under-reporting fast motion (too high) against a lagging estimate that produces a spurious motion blip when the staff is set down (too low). The spin-axis smoother's $\alpha=0.35$ plays the same role for Laban Space: higher reads axis noise as “inconsistent” too readily, lower reads a genuinely wandering spin as falsely focused. The facing hysteresis factor (1.15) sets the $\approx 48^\circ/42^\circ$ dead-band; 1.0 reintroduces boundary flicker, while a much larger factor makes facing changes sluggish to register. The thrust detector's 1.5g threshold and $90^\circ/\text{s}$ gate trade false triggers during forceful spins (too low) against missed soft stabs (too high). None of this has been swept against recorded data; Discussion returns to it alongside the missing perceptual evaluation.

Software: Mapping Architecture

NodeGraph/NodeTypeRegistry: *three node categories - Source (IMU, Laban features), Math (arithmetic, shaping, dynamics, lookup tables), Sink (synth parameters) - each a small typed function registered once. A graph is nodes plus typed wiring, serialised to/from the Extensible Markup Language (XML) as a preset; GraphMappingStrategy makes the graph the only mapping path the synth talks to, with no separate hand-written path.*

Graph Evaluation and Primitive Library. **DAG evaluation.** A patch is a directed graph of typed nodes; NodeGraph::connect provisionally adds an edge and recomputes a topological order with Kahn's algorithm via NodeGraph::recomputeTopoOrder, reverting the edge if the graph fails to fully drain (a closed cycle). Each

audio block, `NodeGraph::evaluate` walks the cached order once, and every node computes

$$y_i = f_{\text{type}(n_i)}(x_{i,1}, \dots, x_{i,m}; \theta_i)$$

where n_i is the i -th node in topological order, $\text{type}(n_i)$ its registered node-type function, $x_{i,1}, \dots, x_{i,m}$ its wired inputs, θ_i its parameters, and y_i its output(s) - so the whole patch is one composed function of the sensor frame, evaluated once per block with no re-sorting on the audio thread.

Primitive vocabulary. Every node type is one small pure function (or, for stateful ones, a function plus a scratch struct): arithmetic (add/subtract/multiply/divide/abs/max/min/equals), shaping (mapRange, clamp, threshold, crossfade, quantizeSteps, sine, atan2, semitonesToHz), lookup tables (a general 1-D table and the 3-boolean-selector table used by the Azimut root table below), and stateful dynamics (one-pole smoother, leaky integrator, retrigger envelope, hysteresis step, derivative, latched smoother).

[Graph::buildAllNodes]

Software: Named Mappings as Graph Presets

Table 2 shows the generalisation claim directly: each of the eleven mappings - Simple, Lead+Drone, Spin Filter, Bowed Chord, the Azimut family (facing-based root table, spin-count filter sweep, axial thrust transients), Martial Effort/Momentum, Spin Voices, and Speed Gate - is built once via `GraphBuilder` and is structurally just a particular wiring of the same node catalog. Counts are read directly off the constructed graphs (`NodeGraph::nodes()`).

Table 2: Node composition of each named-mapping preset.

Preset	Nodes	Source	Math	Sink	Distinct types
Simple	67	31	5	31	21
Lead+Drone	103	26	46	31	28
Spin Filter	107	31	54	22	25
Bowed Chord	116	25	60	31	36
Azimut	126	37	56	33	40
Azimut+	126	37	56	33	39
Azimut Reverb	134	37	62	35	42
Martial Effort	210	46	133	31	41
Martial Momentum	120	37	51	32	35
Spin Voices	117	42	38	37	33
Speed Gate	197	58	107	32	46

Across all eleven presets, only 63 of the catalog's 79 registered node types are ever instantiated - 42 of those source/math primitives, the rest sink writers. A full development cycle of hand-built mapping design, spanning a solo tilt-controlled mapping through the fully Laban-driven Martial and Speed Gate patches, lands inside less than two-thirds of an already small catalog; no preset needed a node type invented specifically for it. That result should be read carefully, though: the catalog itself was not fixed in advance and then tested against eleven independent designs - it grew alongside the eleven mappings, several of which were the reason a given primitive was added in the first place. What Table 2 demonstrates is internal consistency (the resulting catalog is compact enough to re-express every mapping it was abstracted from as a preset, rather than needing a bespoke escape hatch per mapping), not that the catalog would prove sufficient for a gesture vocabulary designed independently of it. Whether a mapping designer outside

this project can build a genuinely new movement-quality mapping from the existing 63 node types without hitting a missing primitive is an open question the current evidence does not settle.

[NodeGraph::nodes] [Graph::Presets::AllPresets]

Azimut-Specific Transformations. Shared by every Azimut-family preset.

Root-note table. The spin-plane/direction/facing root-note table above is not an if/else chain - it is literally an 8-entry lookup table (LUT) indexed by three booleans (isVertical, isCCW, isFacingEast) through the graph's generic 3-selector LUT node.

Root morph. The target semitone value is chased by a variable-rate one-pole filter whose rate is

$$\text{rate} = \max\left(\text{clamp}\left(\frac{\|\omega\|}{2000}, 0.005, 0.2\right), 0.5 \cdot \text{onsetEnv}\right)$$

where `onsetEnv` is a retrigger envelope (decay 0.90) that fires the instant the staff leaves rest - faster motion, or a fresh onset, morphs the root pitch in quicker.

Filter sweep. The spin count drives a phase modulator whose sine output is remapped onto the filter's cutoff range and smoothed:

$$c_{\text{raw}} = \text{mapRange}(\sin(1.5 \cdot \text{spinCount}), [-1, 1], [400, 20000]\text{Hz})$$

cutoff = onePole(c_{raw} , rate=0.03)

where `spinCount` is the unsigned lap counter from Continuous Spin Count above, `mapRange` linearly rescales its argument from the first interval to the second, c_{raw} the resulting unsmoothed cutoff target, and `onePole` the smoothing primitive defined earlier.

[Graph::Presets::buildAzimutCore] [buildSpinCountLpfHz]

Graphical Interface: Live Patching as Performance Practice

The node-graph editor is the instrument's primary authoring surface, not a debug tool: a pannable, zoomable canvas on which each node renders as a coloured card - green for Source, yellow for Sink, neutral for Math - labelled with its category and inline, live-editable parameter fields; typed input/output pins along each card's edges accept click-and-drag connections rendered as cubic-Bézier wires, colour-highlighted while dragging and hit-testable afterward for rewiring or deletion via context menu. New nodes are added from a categorised picker at any canvas position; an auto-layout pass untangles a freshly loaded preset into left-to-right source-to-sink columns. A dirty-state indicator tracks whether the active patch has diverged from its saved preset, with one-click revert - so a performer can improvise a mapping change mid-rehearsal and either keep or discard it, treating the mapping itself as something authored and iterated on at the same level as the sound it produces.

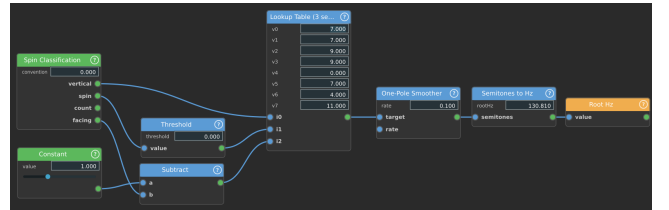


Figure 4: Schematic of the node-graph editor canvas - coloured node cards (green Source, yellow Sink, neutral Math) with typed pins, connected by Bézier-rendered wires.

Software: Synthesis Engine

Every mapping graph, regardless of preset, writes into the same `BoStaffSynth`: a shared four-voice additive engine exposing one mapping-agnostic parameter surface (gain, drive, noise, filter cutoff, pan, reverb send) that sink nodes address by name. Keeping the synthesis engine itself ignorant of which preset is driving it is what lets a graph be swapped or rewired live without reinitialising audio.

Additive Synthesis Engine.

```
[BoStaffSynth::processBlock] [SynthVoice::tick]
```

Harmonic bank. Each of the 4 voices sums 6 phase-accumulated sine partials at integer multiples of a (vibrato- and chorus-modulated) fundamental, each with its own independently smoothed target amplitude.

Waveshaping. A rational soft-clip saturator, applied per-voice when `driveAmt > 0`:

$$y = \frac{x(1 + \text{driveAmt})}{1 + |x(1 + \text{driveAmt})|}$$

where x is the voice's summed harmonic-bank sample before shaping and `driveAmt` the mapping-supplied drive amount.

Resonator. A short feedback comb (1800-sample delay, 0.35 feedback, mixed 0.7 dry / 0.3 wet) is layered onto every voice for a string-like coloration.

Modulation. Vibrato and tremolo are sinusoidal multipliers on pitch and gain respectively:

$$f' = f(1 + \text{depth} \cdot \sin \phi_v) \quad g' = g(1 - \text{depth} \cdot (0.5 + 0.5 \sin \phi_t))$$

where f, g are the fundamental frequency and master gain before modulation, f', g' the modulated values actually used, and ϕ_v, ϕ_t the vibrato and tremolo Low-Frequency Oscillator (LFO) phase accumulators (radians), each advancing by its own rate every sample. A per-voice phase-offset sine (depth 0.003, rate 0.3 Hz) adds chorus detuning; chord changes cross-fade linearly between old and new semitone offsets over 200 ms rather than jumping.

Noise and filtering. An `xorshift32` pseudo-random number generator (PRNG) feeds a one-pole lowpass (coefficient `noiseLpCoef`) before being mixed in at `noiseAmount`; the summed stereo output passes through a JUCE state-variable Topology-Preserving Transform (TPT) lowpass, its cutoff set directly each block from the graph's sink. `lpfCutoffHz` (only range-clamped, not ramped), and an optional Schroeder-style `juce::dsp::Reverb send` whose wet level is ramped through a per-sample `SmoothedValue` to avoid zippering when the active mapping changes.

Pitch/frequency conversion.

$$f = f_{\text{root}} \cdot 2^{\text{semitones}/12}$$

where f_{root} is the mapping's current root frequency in Hz and semitones the signed offset applied to it, giving the equal-tempered result f .

```
[MathHelpers::semitoneRatio] [convertSemitonesToHertz]
```

5 Development Iterations

REMORA's architecture did not arrive fully formed; the node-graph engine described in §4 is the result of three iterations, summarised below, in which generalisation followed rather than preceded hand-built mapping design.

Iteration 1: Sensing Pipeline

The first iteration was firmware reading the IMU and streaming OSC, a minimal receiver on the plugin side, and a single hardcoded tilt-to-pitch mapping. The goal was purely to prove the sensor-to-sound pipeline end to end, not to explore expressivity.

Iteration 2: Hand-Built Mapping Library

The eleven named mappings - Simple, Bowed Chord, Lead+Drone, Spin Filter, Martial Effort/Momentum, Azimut and its variants, Speed Gate, Spin Voices - were then built one at a time as their own C++ classes, each adding a new gesture vocabulary (chords, Laban Effort, world-frame facing, transient detection) on top of the shared `StaffMotionAnalyzer`.

Iteration 3: Generalising into the Node Graph

Once eleven mappings existed as separate classes, a pattern became visible: every one of them was really just arithmetic, shaping, and a handful of stateful primitives wired together in a fixed pattern. The third iteration rebuilt the mapping layer as the small generic graph engine described in §4, porting all eleven prior mappings over as presets rather than deleting them.

6 Evaluation

No performer evaluation has been run yet. This section separates what the current claims actually rest on - implementation-level evidence internal to the system itself - from a protocol for the performer session that would be needed to test the remaining, human-facing parts of those claims. Table 3 summarises this split per claim; the subsections below explain each row.

Table 3: Claims, current evidence, and open questions.

Claim	Current evidence	What is still untested
(1) live graph, no recompile	DAG evaluation/rewiring is implemented and exercised continuously during development (§4)	whether a performer other than the author can rewire it unaided
(2) 11 mappings as presets	Table 2: all eleven hand-built mappings rebuilt as presets from the same catalog	whether a mapping designed independently of the catalog fits without a new primitive
(3) derivation is implemented and magnetometer used throughout development free sessions facing		drift has not been measured against a ground-truth heading; bounded to short sessions by assumption, not measurement
(4) the correction transform is mount-agnostic calibration	mount-independent by construction (§7)	not yet tested across multiple physical mount positions on hardware

Evidence from the Implemented System

Claims (1), (2), and (4) are, at present, supported by the system as built and run by the author during development rather than by an outside observer. The graph evaluator provisionally adds

and removes edges, recomputes a topological order, and drives synthesis every audio block without recompilation or an audio dropout - this is exercised every time a preset is loaded or a wire is changed during development, not a claim about untested code. Table 2's per-preset node counts are read directly off the constructed graphs, so claim (2)'s "eleven mappings, one catalog" result is a property of the code, not a reported impression. Claim (4)'s mount-independence follows from the two-stage calibration derivation (§4) holding for any physically valid sensor placement, which is a property we can state mathematically but have not yet stress-tested by physically remounting the enclosure at several positions and re-running calibration at each. Claim (3) is the weakest of the four on current evidence: the facing heuristic runs correctly in every development session, but "correctly" here means "matched the author's own sense of where the staff was pointing," which is not a substitute for a logged, ground-truth comparison.

Planned Performer Evaluation

To test the parts of these claims that are inherently about performer experience rather than code behaviour - whether the graph is rewirable by someone other than its author, whether a Laban-driven preset reads as more physically legible than a simple one, whether the node picker scales to larger patches - we plan a semi-structured session with performers who have some staff, baton, or movement-flow background but no prior exposure to REMORA. The intended structure: a short calibration walkthrough; free play across a small span of existing presets chosen to range from a minimal mapping to a fully Laban-driven one; a guided patching task (retuning an exposed parameter, then rewiring which motion feature drives a sink); and a closing semi-structured interview. Candidate measures include calibration time, time to complete the guided rewire unassisted, think-aloud commentary during free play, a preference judgement between at least two presets with a reason, and any node-picker search difficulties observed or reported. Each measure is chosen to speak to a specific claim in Table 3 rather than to general usability. This protocol - not results - is what we currently have; running it, with informed consent handled as described in the Ethical Statement, is the immediate next step for this work.

Current Evidentiary Limitations

To be explicit: no human-subject data has been collected for this instrument. Claims (1), (2), and (4) currently rest on implementation-level evidence - the graph's own structure and behaviour under the author's use - not on an independent performer's experience. Claim (2)'s catalog was grown alongside the eleven mappings it reproduces, so its sufficiency for a mapping designed independently of the project remains open (§4). Claim (3) is bounded to short sessions by assumption pending a measurement that has not yet been made. None of the four claims should be read as validated by user testing until the protocol above, or something like it, has actually been run.

7 Discussion

Advantages and Limitations

The generalisation into a graph buys expressiveness and shareability - eleven previously hand-coded mappings collapse into presets of one small dataflow language - at the cost of a patching step that performers used to fixed instruments do not otherwise have to take. The limitations below are ones the current build genuinely carries, not future work described in present tense.

Network deployment is manual. REMORA joins the host device's own Wi-Fi hotspot rather than a dedicated access point, trading range and interference robustness for a one-step pairing setup. That convenience hides a real cost: the hotspot's Service Set Identifier (SSID)/password and the OSC destination IP address are constants compiled into the firmware. A phone hotspot does not guarantee the same IP across sessions, and pairing with a different host network means a different SSID and password entirely - either case currently requires editing the firmware source and reflashing the microcontroller over the USB before the staff will talk to the plugin again. There is no runtime Wi-Fi configuration, no network discovery of the receiving host, and no feedback if the join fails silently.

Single IMU. All orientation and motion-quality features come from one IMU with no magnetometer fusion; heading is deliberately kept relative to the calibration pose rather than magnetic north, since the magnetometer is unreliable this close to the staff's own electronics and typical stage rigs. That keeps facing free of magnetic interference, but it also means REMORA has no absolute reference at all - any yaw drift accumulated since calibration is invisible and uncorrected, and the facing feature is only ever as good as that session's calibration. We have not measured drift quantitatively - no ground-truth heading reference has been logged alongside a session - so rather than report a number we do not have, we explicitly bound claim (3) to session lengths on the order of tens of minutes of active play from a single calibration, matching the durations used during development. That bound is an assumption carried over from informal use during development, not a measured guarantee: nothing in the current implementation prevents drift from growing unboundedly over a longer, uninterrupted rehearsal or set, and no re-calibration-on-demand gesture currently exists to correct it mid-performance. No quantitative or perceptual evaluation of facing or calibration accuracy has been carried out yet; it is one of the open items the planned performer evaluation (§6) does not yet address either, since ground-truth heading logging would need to be added to the protocol first.

Graph interface complexity. The node-graph editor is expressive for whoever is building or modifying a mapping, but it exposes that same complexity to a performer who only wants to select and play an existing preset: there is no simplified performance-only view that hides the graph until it's asked for. As it stands, the interface serves instrument-builders and patch-authors more directly than first-time performers, and whether that cost is acceptable is exactly the kind of question the still-missing formal user study would need to answer.

Remaining software robustness gaps. A few implementation-level issues would need attention before REMORA is exposed to inputs beyond the author's own testing: incoming OSC messages

are read without confirming their argument types match what the receiver expects, dropped or reordered packets are not detected as such, and the handoff of a sensor snapshot from the network thread to the audio thread retries in a tight spin loop rather than degrading gracefully under contention.

Hardware. The enclosure carries a 2000 mAh LiPo battery, sized to comfortably outlast a single rehearsal or set at the firmware’s steady 100 Hz sampling/transmission rate based on the ESP32-S2’s typical current draw, but runtime under REMORA’s actual firmware has not been measured: no discharge test has been logged to date. That measurement - ideally several runs to a defined cutoff voltage, reported with variance rather than a single number - is needed before any runtime claim can be made, and before the battery can be sized with confidence for a full day of back-to-back performances without a recharge.

Future Improvements

Wireless transport. The hotspot-join scheme in §7 could be replaced with ESP-NOW: the staff-mounted ESP32 would send its sensor packets directly, peer-to-peer, to a second ESP32 tethered over USB to the host computer, which forwards them to the plugin over serial or a loopback OSC bridge. That removes the SSID/password/IP baked into firmware entirely - ESP-NOW pairs by MAC address rather than joining a network - and sidesteps the “new hotspot means reflash” problem, at the cost of adding a second microcontroller to the setup.

Other extensions. Gesture-triggered mode switching, a merged rest/motion mapping, a two-IMU dual-tip configuration, further Laban Effort mapping targets, machine learning based discrete gesture recognition, and eventually exposing the node graph itself as a shareable/community patch format.

8 Conclusions

REMORA is a wireless, IMU-driven *bô* staff instrument, but its contribution is architectural rather than being one more fixed gesture-to-sound mapping: the same small dataflow language - source, math, and sink node primitives, evaluated as a DAG once per audio block and rewirable without recompiling - is expressive enough to reproduce all eleven mapping strategies developed over the course of the project as ordinary, live-editable presets (Table 2), from a minimal tilt-to-pitch patch to the fully Laban-driven Martial and Speed Gate mappings. That claim is grounded in an explicit, code-traceable account of mount-agnostic calibration, orientation tracking, and movement-quality extraction (§4), not asserted in the abstract alone.

What remains uncertain is exactly what §6 makes explicit: whether a performer other than the author finds the graph rewirable and the Laban-driven presets more legible, whether the 63-type primitive catalog holds up against a mapping designed independently of it, and how much heading drift accumulates beyond the session lengths used during development are all open questions that implementation-level evidence alone cannot settle. The most important next step for this work is therefore not another architectural extension but running the performer evaluation protocol outlined in §6, so that the claims this paper makes about live-patchable, Laban-grounded gestural control can be tested against use by someone other than the instrument’s own builder.

Acknowledgments

This work was carried out during the first author’s internship at CREATE, Aalborg University, Denmark.

9 Ethical Statement

Accessibility. REMORA assumes a range of mobility and two-handed grip strength consistent with swinging a *bô* staff; it is not currently adapted for performers who cannot support or swing a 1–1.5 kg full-length staff, and no alternative mounting or reduced-effort variant has been explored. This is a real limitation of the instrument’s physical form, not merely of the present build. That said, the skill barrier is not uniform across the preset catalog: presets such as Simple ask only for basic tilt and roll and do not assume any prior staff-handling technique, while the Laban-driven presets (Martial, Azimut) reward accumulated skill with the instrument’s physical vocabulary. Because every preset is authored in the same shared patching language (§4), a facilitator or performer can also build or select a reduced-effort mapping for someone without a staff background, rather than being limited to the eleven built-in presets. This is a partial mitigation on the skill axis, not a solution to the physical one above.

Environmental impact. The staff-mounted hardware consists of a 3D-printed/OpenSCAD-parametrised enclosure, a rechargeable LiPo battery, an ESP32-S2 microcontroller, and an MPU-9250 IMU. The enclosure’s zip-tie mounting (§7, The Build) avoids adhesives, so it can be removed and the plastic reused or recycled without damaging the staff; the electronics are similarly reusable across builds since nothing is glued in place. No lifecycle or embodied-carbon analysis of the components has been performed.

Socio-economic fairness. REMORA is built entirely on free/libre and open-source tools - JUCE, PlatformIO, and OpenSCAD - and the microcontroller and IMU are inexpensive, widely available commodity parts. This keeps the reproduction cost and toolchain barrier low relative to proprietary DMI development environments, which we consider a genuine point in the instrument’s favour under the NIME Code of Practice’s socio-economic fairness criterion.

Human participants. No human-participant data has been collected to date; all reported observations of the system in use are the first author’s own. The performer evaluation outlined in §6 has not yet been conducted. When it is run, participants will give informed consent before any session, be referred to only by anonymised identifiers, and be free to withdraw at any point, consistent with this statement’s other commitments.

Use of Artificial Intelligence. Per the NIME Code of Practice’s guidance on Artificial Intelligence use, we disclose that a locally-run large language model (Mistral 7B) was used to help draft and edit portions of this manuscript’s prose under the first author’s direction and review. This disclosure applies only to the writing of this paper: REMORA’s firmware, plugin, and mapping code were designed and written entirely by the first author without AI code generation.

References

- [1] Andrew Belt and VCV. 2017. VCV Rack: Open-Source Virtual Modular Synthesizer. <https://vcvrack.com>. Software, first released September 2017.
- [2] Peter Bennett, Nicholas Ward, Sile O’Modhrain, and Pedro Rebelo. 2007. DAMPER: A Platform for Effortful Interface Development. In *Proceedings of*

- the International Conference on New Interfaces for Musical Expression (NIME)*. New York, USA. https://www.nime.org/proceedings/2007/nime2007_273.pdf
- [3] Antonio Camurri, Shuji Hashimoto, Matteo Ricchetti, Andrea Ricci, Kenji Suzuki, Riccardo Trocca, and Gualtiero Volpe. 2000. EyesWeb: Toward Gesture and Affect Recognition in Interactive Dance and Music Systems. *Computer Music Journal* 24, 1 (2000), 57–69. doi:10.1162/014892600559182
 - [4] Joseph Malloch and Marcelo M. Wanderley. 2007. The T-Stick: From Musical Interface to Musical Instrument. In *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*. New York, USA, 66–69. doi:10.1145/1279740.1279751
 - [5] Teresa Marrin and Joseph Paradiso. 1997. The Digital Baton: A Versatile Performance Instrument. In *Proceedings of the International Computer Music Conference (ICMC)*. Thessaloniki, Greece. <https://quod.lib.umich.edu/i/icmc/bbp2372.1997.083>
 - [6] Teresa Marrin and Rosalind W. Picard. 1998. The “Conductor’s Jacket”: A Device for Recording Expressive Musical Gestures. In *Proceedings of the International Computer Music Conference (ICMC)*. Ann Arbor, MI, USA. <https://www.media.mit.edu/publications/the-conductors-jacket-a-device-for-recording-expressive-musical-gestures-2/>
 - [7] Sangbong Nam. 2013. Musical Poi (mPoi). In *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*. Daejeon, Korea, 148–151. https://www.nime.org/proceedings/2013/nime2013_254.pdf
 - [8] Miller Puckette. 1988. The Patcher. In *Proceedings of the International Computer Music Conference (ICMC)*. San Francisco, CA, USA, 420–429.
 - [9] Miller Puckette. 1996. Pure Data: Another Integrated Computer Music Environment. In *Proceedings of the Second Intercollege Computer Music Concerts*. Tachikawa, Japan, 37–41.
 - [10] Miller Puckette. 2002. Max at Seventeen. *Computer Music Journal* 26, 4 (2002), 31–43. doi:10.1162/014892602320991356
 - [11] Jan C. Schacher. 2010. Motion to Gesture to Sound: Mapping for Interactive Dance. In *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*. Sydney, Australia, 250–254. https://www.nime.org/proceedings/2010/nime2010_250.pdf
 - [12] Nicholas Ward, Kris Penfield, Sile O’Modhrain, and R. Benjamin Knapp. 2008. A Study of Two Thereminists: Towards Movement Informed Instrument Design. In *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*. Genova, Italy. https://www.nime.org/proceedings/2008/nime2008_117.pdf